

Task 1 - Hello World

The first task will be to implement a simple “Hello World” program. In order to be stateful, we will require the program to have a capsule with a state. The capsule will be called *Main* and the state will be called *Hello*. When the program starts, *Main* will change state to *Hello*, then print “Hello World” and then terminate. This can be illustrated in figure 1. The expected output from the program can be seen in figure 2.

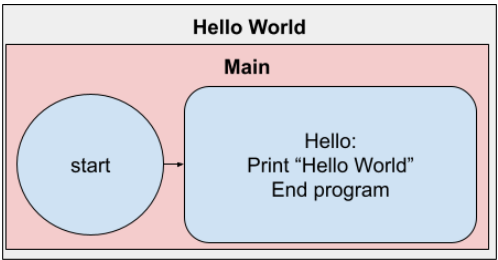


Figure 1 - Hello World Program



Figure 2 - Hello World output

Task 2 - Hello Timer

The second task will continue from the Hello World program and introduce a timer. Main will have 2 states: Waiting and End. When the program starts it will go into the Waiting State, start a timer of 5 seconds and print “Hello World!”. When the timer expires, Main will go into the End state, print “Goodbye World!” and end the program. This can be illustrated in figure 3. The expected output from the program can be seen in figure 4.

The timer shall be run on a separate thread.

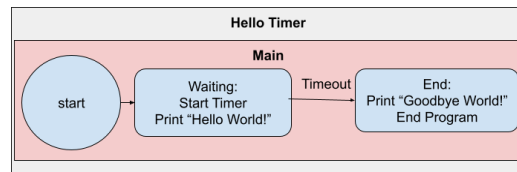


Figure 3 - Hello Timer Program

```
Hello World!
Goodbye World!
```

The image shows a terminal window with a black background and white text. The first line displays 'Hello World!' and the second line displays 'Goodbye World!'.

Figure 4 - Hello Timer output

Task 3 - Digit Counter

The third task continues from the Hello Timer program and will introduce a state machine with an infinite loop and a timer that repeatedly sends timeouts. Main will have 10 states: Zero, One, Two, Three, Four, Five, Six, Seven, Eight, Nine. When the program starts a timer will start that repeatedly sends timeout messages. When Main receives a timeout, a transition will happen and the number corresponding to the state will be printed. This is illustrated in Figure 5. One example of output from the program can be seen in figure 6.

The interval between each timeout shall be set in Main's constructor. For testing purposes this can be set to 1 second when calling the constructor.

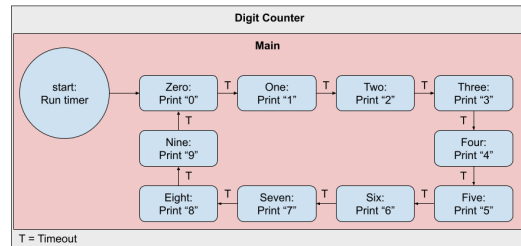


Figure 5 - Digit Counter Program

```
0123456789012345678901234
```

Figure 6 - Digit Counter output

Task 4 - Multi-Digit Counter

The fourth task continues from the Digit-Counter program and will introduce multiple capsules and communication between them. There will be 1 Main capsule, 3 Digit capsules and 1 Printer capsule. The state diagrams can be seen in figure 7. The first digit will represent a number's ones value, the second digit will represent the tens value and the third digit will represent the hundreds value. One example of output from the program can be seen in figure 8.

The main capsule will be responsible for sending Inc-messages to the first digit (Inc standing for increment). The interval for each increase will be set in Main's constructor. For testing purposes we can set this value between 1 millisecond and 1 second.

The Digit capsule will be similar to the main capsule in the previous task. It will make transitions when it receives an Inc-message instead of timeouts. It will also send an Inc message when it gets an overflow (transitions from Nine to Zero). The first digit's overflow will send an Inc-message to the second digit. The second digit's overflow will send an Inc-message to the third digit. The third digit's overflow will not be connected and not send an Inc message. The Digit capsule will no longer print the state it is in. It will be able to receive a request for its current digit and respond with the digit corresponding to the state it is in.

The printer capsule will handle the printouts. It will repeatedly request the value of the digits. After receiving all the digits, it will print the number. All the numbers will be printed, including leading zeros. The interval between each print cycle will be set in the Printer's constructor. For testing purposes, this can be set to 0.5 seconds.

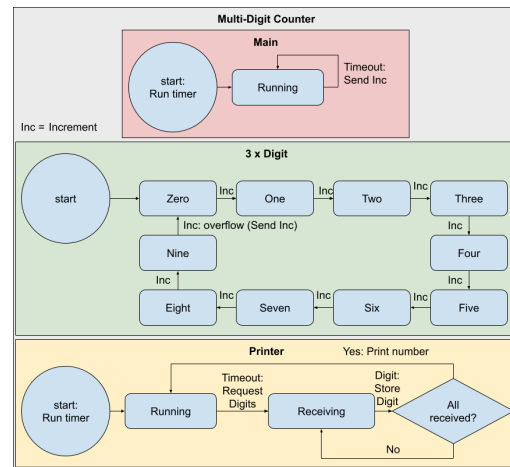


Figure 7 - Multi-Digit Program

```

029 057 085 114 142 170 199 227 255 284 312 330 369 397 425 454
482 500 539 567 595 624 652 680 708 737 765 793 822 840 878 907
935 963 992 010 048 077 105 133 162

```

Figure 8 - Multi-Digit output

Task 5 - Dynamic Digit Counter

The fifth task will continue from the Multi-Digit counter program. The program shall be able to handle any amount of digits. It starts with one digit and whenever it needs a new digit, a new digit capsule will dynamically be created and connected to the previous most significant digit. The state diagram can be seen in figure 9. One example of output from the program can be seen in figure 10.

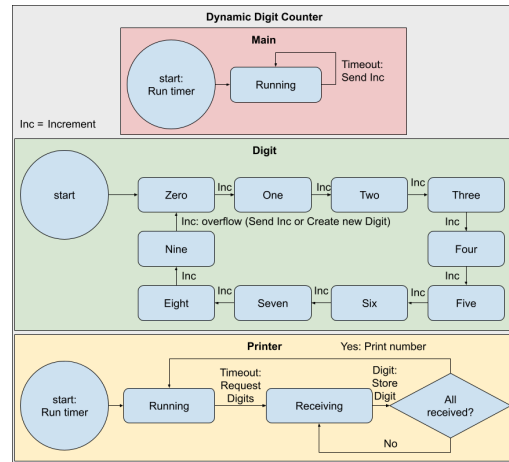


Figure 9 - Dynamic Digit Counter Program

```

19 37 56 74 93 111 129 148 166 185 203 221 240 258 277 295 313 332
350 369 387 405 424 442 461 479 498 516 534 553 571 580 608 626
645 663 682 700 718 737 755 774 792 810 829 847 866 884 903 921
939 958 976 995 1013 1031 1050 1068
  
```

Figure 10 - Dynamic Digit Counter output

Task 6 - Hello SFML

Make a new program that opens a window and displays the text “Hello SFML”. When the user closes the window the program shall end. The state diagram can be seen in figure 11. The result with the program running can be seen in figure 12.

A library called Gui will be used for this task. Gui is built with SFML and will be provided before starting, as well as the necessary font.

The dimensions of the window shall be set in Main’s constructor. For testing purposes the width can be set to 800 and the height to 500.

The frame rate (frames per second) shall also be set in Main’s constructor. Note that frame rate is the inverse of the time interval between frames. For testing purposes the frame rate can be set to 30 fps.

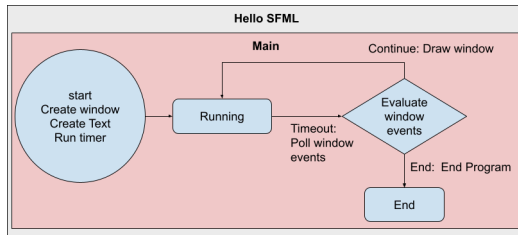


Figure 11 - Hello SFML Program

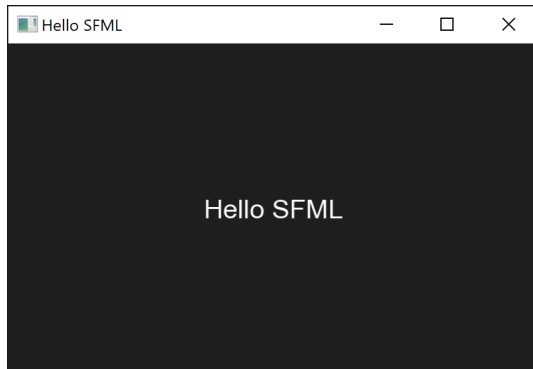


Figure 12 - Hello SFML running

Task 7 - SFML Digit Counter

The seventh task will combine the Dynamic Counter program with the Hello SFML program and add some user input. Instead of the number being printed, the number will be displayed in a window. The printer capsule is replaced by a Gui capsule and the state diagrams can be seen in figure 13. One example of the program running can be seen in figure 14.

The Gui capsule will handle user inputs (when polling window events). When the user presses P, a message will be sent to the main capsule to play or pause. When the user presses space bar, then a message will be sent to the 1st digit to increase.

The main capsule will have a new state called Pause and it will start in the pause state. It will toggle between play and pause when it receives a message from the Gui (by user pressing P). Main's timer will only run in its running state and will only send Inc messages, while it is running.

The Gui capsule shall set the frame rate and the windows dimensions in its constructor. For testing purposes the frame rate can be set to 30 fps, the window width to 800 and the window height to 500.

The main capsule shall set the time interval between its inc messages in its constructor. For testing purposes this can be set between 1 millisecond and 1 second.

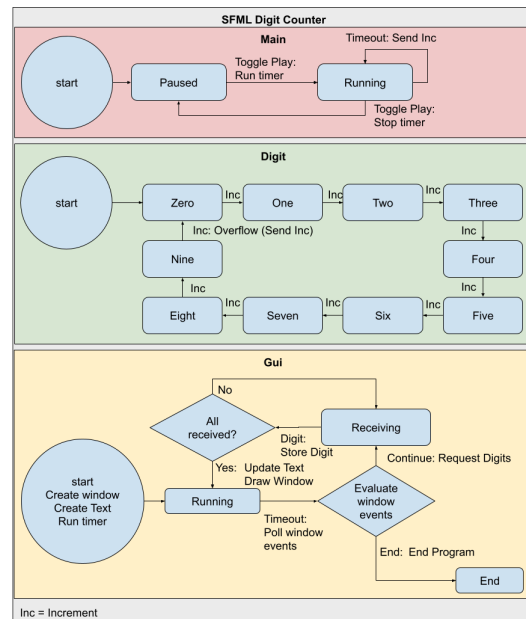


Figure 13 - SFML Digit Counter Program

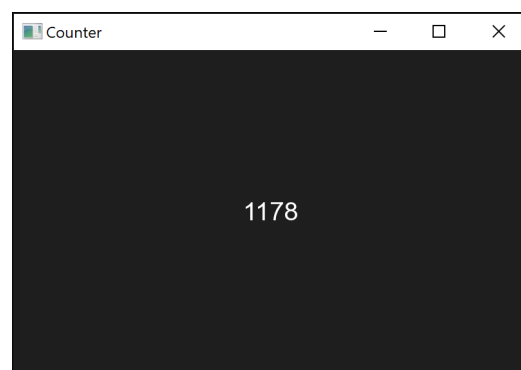


Figure 14 - SFML Digit Counter running

Task 8 - Interactive Grid

The eighth task is to create a grid that is displayed in a window with cells that can be toggled. The state diagram can be seen in figure 15. One example of the program running can be seen in figure 16.

Each cell in the grid will be represented by a capsule called Cell. The Cell will have 2 states: Alive and Dead. It will toggle between these states when it receives a toggleMessage. It will also send a message to the Gui when it changes state.

The Gui capsule will draw the grid in a window, covering the whole window. A dead cell will be drawn as white. An alive cell will be drawn as black. The Gui will update the color of a cell when it receives a status message from that cell. The Gui capsule does not need to request the values of the cells, since the cells send their status when they change state.

User input will be handled in the Gui capsule when it polls window events. When the user clicks on a cell a message will be sent to that cell to toggle it. When the user presses the spacebar, a message will be sent to all cells to toggle them. When the user presses P, a togglePlay message will be sent to the Main capsule.

The Main capsule is similar to the main capsule in the previous task. The difference being that it will send toggleMessages to all cells instead of sending an inc message to a digit.

The number of rows and columns in the grid will be used when creating the cells and will be used in the Gui's constructor. For testing purposes the number of columns can be set to 20 and the number of rows to 6.

The windows dimensions should be set in the Gui's constructor. For testing purposes the width can be set to 800 and the height to 500.

The windows frame rate should be set in the Gui's constructor. For testing purposes this can be set to 30.

The time between each toggle message in the Main capsule should be set in Main's constructor. For testing purposes this can be set to 0.5 seconds.

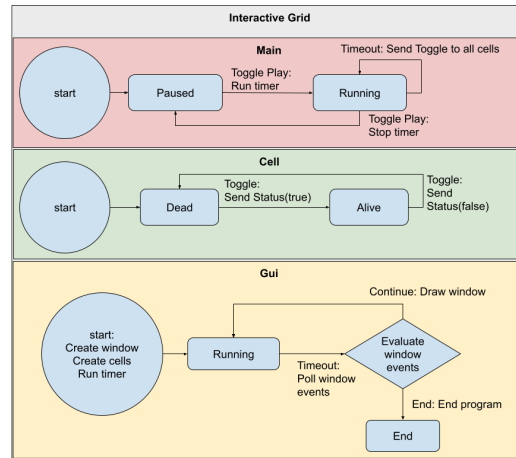


Figure 15 - Interactive Grid Program

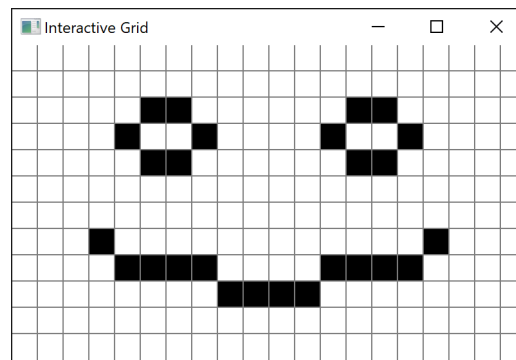


Figure 16 - Interactive Grid running

Task 9 - Neighbor Counter

The ninth task will continue from the Interactive grid program. Each cell shall keep track of the number of alive neighbors they have and the window should be able to write this number inside each cell. The state diagram can be seen in figure 17. One example of the program running can be seen in figure 18.

A cell will be considered a neighbor if it is alive and either orthogonal or diagonal adjacent to a cell. When a cell changes state (from dead to alive or alive to dead), the cell should not only send a message to the Gui, but also send a message to its neighbors. When a cell receives a message that a neighbor is born, it will increase its neighbor counter by one. When a cell receives a message that a neighbor has died, it will decrease its neighbor counter by one. When the number of neighbors a cell has changes, it will send a message to the Gui with the amount of neighbors it has.

The Gui will have a flag called `displayNeighborCounters`. The `displayNeighborCounters` will indicate whether the counters should be shown. If it is set to true the number will be printed on the cells. The gui shall be able to handle incoming messages to update those numbers. When writing, if the cell is white, the number should be black and if the cell is black the number should be white.

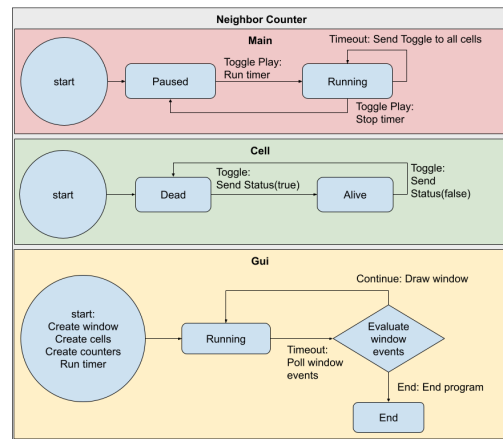


Figure 17 - Neighbor Counter Program

0	0	0	0	0	0	0	0	0	0	0	0	1	2	1	0	0	0	0
0	0	0	1	2	2	1	0	0	1	2	2	2	3	2	2	0	0	0
0	0	1	2	2	2	2	1	1	2	2	2	2	4	2	3	0	0	0
0	0	1	2	5	5	2	1	1	2	5	5	2	3	2	4	2	1	0
0	0	1	2	2	2	2	1	1	2	2	2	2	3	3	3	2	1	0
0	0	0	1	2	2	2	2	2	2	2	2	1	0	2	3	5	2	1
0	0	0	0	0	0	2	3	3	2	0	0	0	0	1	2	2	2	1
0	0	1	1	1	0	2	3	3	2	0	1	1	1	0	1	1	1	0
0	0	1	1	2	1	1	2	2	1	1	2	1	1	1	2	3	3	2
0	0	1	2	2	3	3	3	3	3	3	2	2	1	2	2	4	4	5
0	0	0	1	2	2	2	2	2	2	2	1	0	3	4	8	7	8	5
0	0	0	0	1	2	3	3	3	3	2	1	0	0	2	2	4	4	5

Figure 18 - Neighbor Counter running

The flag for whether to display the number of neighbors should be toggled by the user pressing n.

Task 10 - Game of Life

Continue with the Neighbor counter program. Implement Conway's Game of Life. The cells should be able to handle tick-messages, which follows the game of life's logic. The new state diagrams can be seen in figure 19. One example of the program running can be seen in figure 20.

When the user presses space bar or the main capsule gets a timeout, then tick messages should be sent to all the cells instead of toggle messages. A toggle message should still be sent when clicking on a cell.

If an alive cell receives a tick message when it has either fewer than 2 alive neighbors or more than 3 neighbors, then it dies. If a dead cell receives a tick message when it has exactly 3 neighbors alive, then it becomes alive.

Each tick update is supposed to be made synchronised. One cell being updated before another cell for the next generation should not change the outcome. Therefore each cell will have a sync flag that changes between true and false between each tick. This sync flag will be sent in the status message between cells.

Each cell will have 2 neighbor counters. One currentNeighborCounter and one nextNeighborCounter. Whenever it receives a status update, then if it is synched, both counters are updated. If they are not synched, then only the nextNeighborCounter is updated. Whenever the sync is updated (when a cell receives a tick), then the currentNeighborCounter is set to the same as the nextNeighborCounter.

The neighbor counters should by default not be shown, but should still be able to be toggled by the user pressing n.

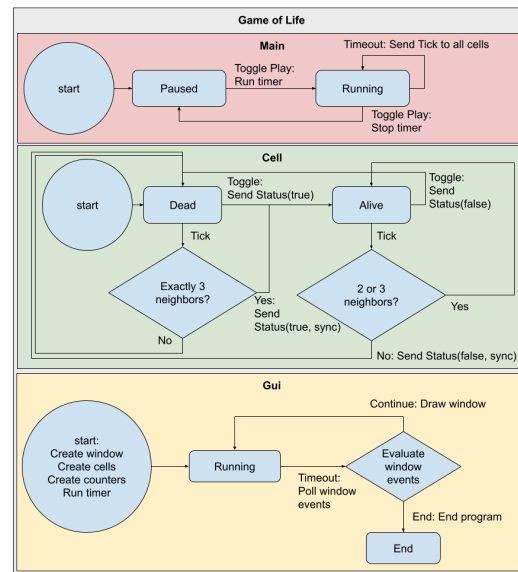


Figure 19 - Game of Life Program

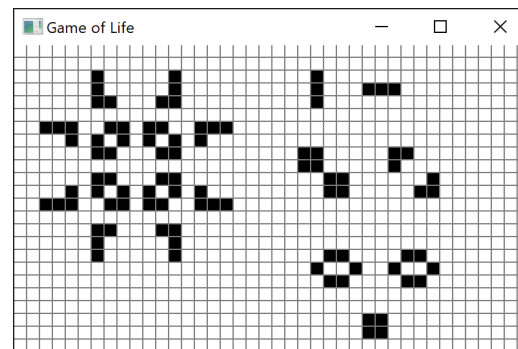


Figure 20 - Game of Life running